

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Computer Science and Engineering
B.Tech II Semester-III
CS231 – Object Oriented Programming Practical Assignment
Lab Assignment - 7

1. Define a class `GenericStorage` with a private `void*` member variable `data`. Implement a constructor that takes a `void*` parameter to initialize `data`. Add methods:
 - `void setData(void* newData)` to update the pointer.
 - Template method `T getDataAs()` to cast and return the data as type `T`.
In `main()`, create an instance of `GenericStorage`, store an integer (e.g., 42), retrieve it as `int`, then store a string (e.g., "Hello"), and retrieve it as `string*`.
2. Define a class `SafeObject` with a private pointer to an integer (`int* ptr`). Implement a constructor that initializes `ptr` to `nullptr`. Add methods:
 - `void allocate(int value)` to dynamically allocate memory and set the value.
 - `int getValue()` that checks if `ptr` is null; if null, return 0 and print "Null pointer detected"; else return `*ptr`.
 - `void deallocate()` to free the memory and set `ptr` to `nullptr`.
In `main()`, create an instance, call `getValue()` before allocation (should handle null), allocate with value 100, get the value, then deallocate and call `getValue()` again.
3. Define a class `ResourceManager` with a private `int*` resource. Implement a constructor that allocates memory for resource and sets it to a value (e.g., 200). Add a method `int* getResource()` that returns the pointer. Add a destructor that deallocates resource.
In `main()`, create an instance of `ResourceManager`, get the resource pointer, print its value, then let the object go out of scope (causing destructor to run), and attempt to access the pointer afterward.
4. Define a class `UninitializedPtrDemo` with a private `int* wildPtr` (do not initialize in constructor initially). Add a method `void initialize(int value)` that allocates memory for `wildPtr` and sets the value. Add a method `int getValue()` that returns `*wildPtr` (without checks initially).
In `main()`, create an instance, attempt to call `getValue()` before `initialize()` (this demonstrates wild pointer—undefined behavior).
Modify the class to initialize `wildPtr` to `nullptr` in the constructor, add a check in `getValue()` for null, and only dereference if not null.
Demonstrate both versions.
5. Define a class `DynamicArray` with private members: `int* arr` and `int size`. Implement a constructor that takes an `int n`, uses `new` to allocate an array of size `n`, and initializes elements to 0. Add methods:
 - `void setElement(int index, int value)` to set `arr[index] = value` (with bounds check).
 - `int getElement(int index)` to return `arr[index]` (with bounds check).
 - Destructor to use `delete[]` to free the array.
In `main()`, create an instance with size 5, set elements (e.g., 1 to 5), print them, then create another instance and let it go out of scope to trigger destructor.
Handle bad allocations with try-catch for `std::bad_alloc`.
6. **Library Book Management** Create a class `LibraryBook` to manage books dynamically in a digital library system.
Create a class `LibraryBook` to manage books dynamically in a digital library system.
Context:
In a university's e-library, book records are managed using pointers. Improper memory handling (like dangling, wild, or null pointers) may lead to incorrect access of book details or even crashes.

Requirements:

- Provide default and parameterized constructors to initialize book title (string) and price (double).
- Use a **wild pointer** to demonstrate uninitialized access to a book object, and later fix it by initializing it with ***nullptr***.
- Demonstrate a **null pointer** case where a pointer is explicitly set to ***nullptr*** before checking for safe access.
- Use a **void pointer** to point to both book title and book price, then cast it properly when retrieving values.
- Show a **dangling pointer** scenario by deleting a book object but still trying to access it, then fix it by assigning ***nullptr***.
- Apply **new** to dynamically allocate memory for a book, and **delete** to safely release it.
- Add a **const** member function ***display()*** to show book details safely.

Sample Input:

```
LibraryBook *b1 = new LibraryBook("C++ Basics", 499.0);
b1->display();
```

```
LibraryBook *b2; // Wild pointer (uninitialized)
b2->display(); // unsafe → fix by initializing nullptr
```

```
void* vp;
vp = b1; // pointing to object
// cast and access details
```

```
delete b1; // Dangling pointer now
b1->display(); // unsafe → fix by setting b1 = nullptr
```

Output:

Book Title: C++ Basics, Price: 499

Error: Wild pointer accessed before initialization!

Error: Null pointer detected, cannot display book details.

Book details accessed using void pointer → Title: C++ Basics

Error: Dangling pointer! Book memory already released.

7. Heterogeneous Container using Void Pointers

Create a class **HeteroContainer** that can store an array of **void*** pointers to hold different data types.

- Implement methods:

- **void add(void* item)** → to insert elements.

- **template<typename T> T get(int index)** → retrieves and typecasts the element.

- In main(), store at least:

- An **int**

- A **float**

- A **string**_(as string*)

- Retrieve and print them using proper typecasting.

- Show what happens if the wrong type is requested (undefined behavior)

(Concepts: void pointer arrays, type safety issues, runtime errors)

8. Null Pointer Safe Linked List

Design a singly linked list using a **Node** class where next is a pointer.

- Ensure that all **next** pointers are initialized to **nullptr** in constructors.
- Implement:
 - **void insert(int value)**
 - **void display()**
 - **int search(int value)** (return index or -1)
- In **main()**, show that:
 - Without null initialization, the list leads to garbage traversal.
 - With proper initialization, null pointer safely terminates traversal.

(Concepts: null pointer as sentinel, avoiding garbage links, safe dynamic structures)

9. Dangling Pointer with File Buffer

Create a class **FileBuffer** that:

- Dynamically allocates a buffer (**char* buf**) for file content.
- Reads data from a text file into the buffer.
- Provides a **char* getBuffer()** method.
- Destructor should **delete[] buf**.

In **main()**:

- Create a **FileBuffer** object, read data, get the buffer pointer, print it.
- After the object goes out of scope, try to print the buffer again (dangling pointer).
- Modify the program to copy the buffer before object destruction to solve dangling pointer.

(Concepts: dangling pointer, destructor behavior, safe memory ownership)

10. Wild Pointer in Dynamic Matrix Allocation

Write a class **Matrix** that manages a 2D matrix with dynamic allocation using double pointers (**int****).

- Constructor should allocate memory for rows and columns.
- If pointers are not initialized properly (wild pointer case), accessing elements leads to undefined behavior.
- Add proper initialization to **nullptr** and allocate safely with **new**.
- Add methods:
 - **void set(int r, int c, int val)**
 - **int get(int r, int c)**
- In **main()**, demonstrate:
 - The effect of using wild pointers (uninitialized **int****).
 - The corrected version with safe initialization.

(Concepts: wild pointer, multi-level pointer allocation, memory safety)

